Aula 59 — Integração e Entrega Contínua (CI/CD)

Computação em Nuvem e Virtualização

Ricardo Pasquati Pontarolli

Técnico em Redes de Computadores · IFSP Boituva · pasquati@ifsp.edu.br · v44

Objetivos da Aula

- Compreender **CI**, **CD** e a diferença entre entrega e deploy.
- Conhecer as **etapas de um pipeline**.
- Identificar **ferramentas** de CI/CD.
- Entender boas práticas de pipelines.

Integração Contínua (CI)

- Integrar o código **frequentemente** (várias vezes ao dia).
- Cada integração dispara **build e testes automatizados**.
- Detecta erros **cedo**.
- Requer **versionamento** (Git) e **testes**.

Entrega × Deploy Contínuo

- **Continuous Delivery:** pronto para produção, **deploy manual**.
- **Continuous Deployment: deploy automático** em produção.
- A escolha depende de **risco e maturidade**.
- Ambos dependem de **testes confiáveis**.

Etapas de um Pipeline

1. **Checkout** do código.
2. **Build** (compilar/empacotar).
3. **Testes** (unitários, integração).
4. **Análise** de qualidade/segurança.
5. **Deploy** (staging/produção).

Pipeline como Código

```yaml
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: echo "compilar"
      - run: echo "testar"
      - run: echo "publicar"
```

- O pipeline é **versionado** com o código.
- Reproduzível e revisável.

Ferramentas de CI/CD

- **GitHub Actions, GitLab CI**.
- **Jenkins** (servidor de automação).
- **AWS CodePipeline, Azure DevOps**.
- Integração com **contêineres e IaC**.

Estratégias de Deploy

- **Rolling:** substituição gradual.
- **Blue-green:** dois ambientes, troca de tráfego.
- **Canary:** liberar para uma fração dos usuários.
- Reduzem **risco** de indisponibilidade.

Boas Práticas

- **Testes automatizados** confiáveis.
- **Pipelines rápidos** e feedback curto.
- **Segredos** gerenciados com segurança.
- **Monitoramento** pós-deploy.
- **Rollback** fácil.

Dica

Um bom pipeline é **rápido e confiável**: é a espinha dorsal da entrega contínua. Deploy sem teste é risco, não agilidade.

Resumo da Aula

- **CI** integra e testa continuamente.
- **CD** entrega/deploya automaticamente.
- Pipelines têm etapas de **build, teste e deploy**.
- Estratégias como **blue-green e canary** reduzem risco.

Atividade Rápida

- Diferencie **entrega** e **deploy** contínuo.
- Liste as etapas de um **pipeline**.
- Explique a estratégia **blue-green**.

Referências

- GitHub Actions / GitLab CI — *Documentação*.
- *Continuous Delivery* (Humble & Farley).

Próxima Aula

- **Aula 60:** Prática — pipeline CI/CD.